

Chapter 3: Methodology

Arthiga Karthigesu

1 Overview of the research design

This chapter specifies the method used to answer the research question: whether a lineage-aware hierarchical classifier, combined with an imbalance-robust training objective, improves fine-grained classification of single peripheral blood cell images — particularly for rare cell types — relative to a flat transfer-learning classifier trained on the same data.

The design is comparative and ablative. A single shared implementation trains every model variant, and the variants differ only in the values of a configuration object, never in code path. This matters because the effects under investigation are small: differences between the arms are on the order of 0.005 to 0.07 in macro F1, so any incidental divergence between a baseline and a proposed model — a different schedule, a different augmentation policy, a differently seeded split — would be large enough to manufacture or destroy the result. Holding everything constant except the factor under test is therefore not a matter of tidiness but a precondition for the comparison being interpretable at all.

Figure 1 shows the pipeline end to end. Images pass through a deterministic preprocessing stage into a pretrained backbone shared by two classification heads; a single composite loss backpropagates through both heads into the backbone; and the trained model is evaluated with hierarchical error analysis and Grad-CAM saliency.

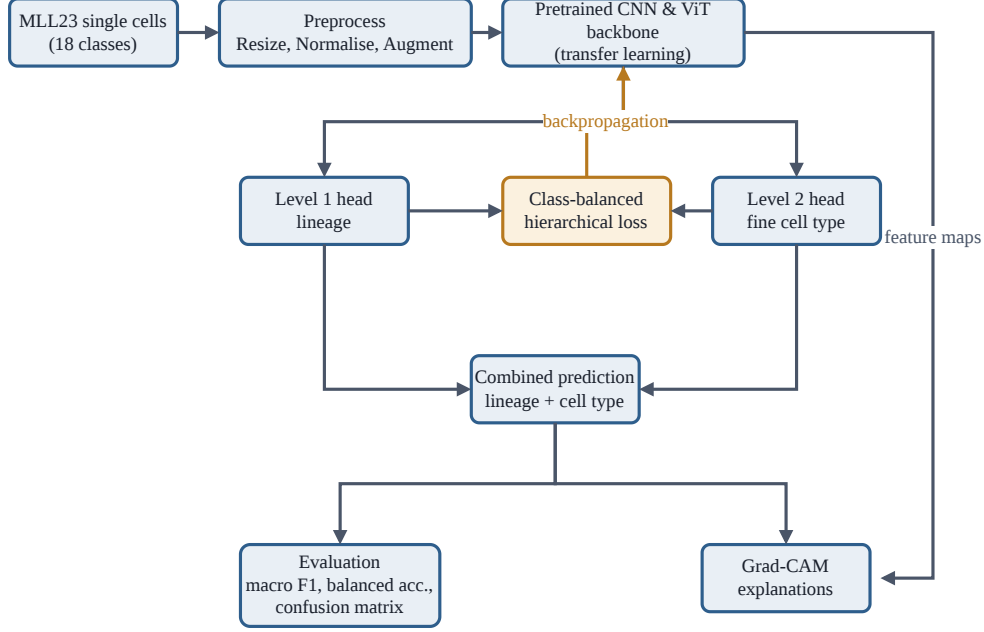


Figure 1: The proposed pipeline. One backbone feeds a lineage head and a fine cell-type head; a class-balanced hierarchical loss couples them and backpropagates into the shared trunk. Evaluation and Grad-CAM both read from the trained network.

2 Dataset

2.1 The MLL23 corpus

The study uses the MLL23 peripheral blood single-cell dataset, published as a data descriptor in *Scientific Data* (2025) and distributed through Zenodo under DOI 10.5281/zenodo.14277609. It contains 41,621 Pappenheim-stained single nucleated-cell images at 288×288 pixels (corresponding to a $25 \mu\text{m} \times 25 \mu\text{m}$ field), each annotated by expert cytomorphologists into one of 18 classes.

Archives are downloaded, MD5-verified against the published checksums, and extracted into one directory per class. A manifest is then built and the realised per-class image counts are asserted against the counts reported in the data descriptor. This verification is not ceremonial: a silently truncated download or a mis-mapped class directory would corrupt every downstream result while leaving a pipeline that still runs, and the class counts are the only external ground truth available to detect it.

Figure 2 shows one representative image from each of the 18 classes after preprocessing. The visual difficulty of the task is apparent directly: the myeloid maturation stages differ by gradual changes in nuclear shape, chromatin condensation and cytoplasmic granularity rather than by any categorical feature, and several lymphoid types are separated by subtle differences in nuclear contour and cytoplasm volume.

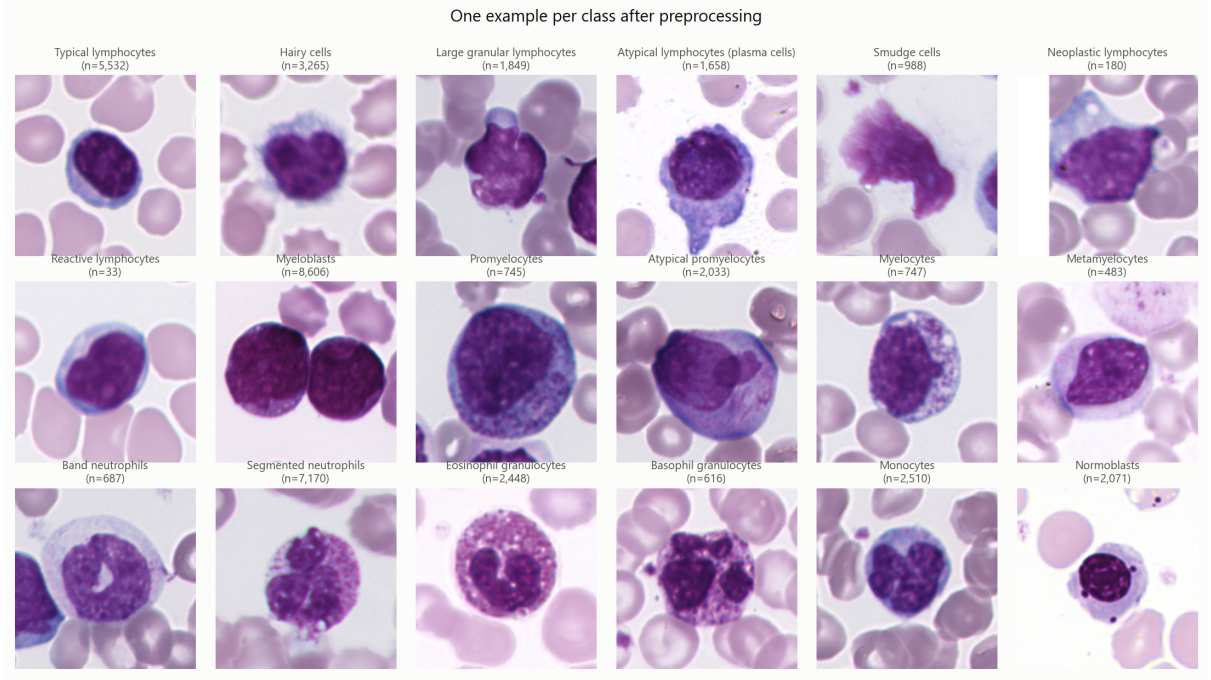


Figure 2: One representative image per class after preprocessing, with corpus counts. Classes are shown in class-index order; within the myeloid branch this is the maturation continuum. The gradual morphological transition between adjacent stages is the source of the within-lineage confusion the study measures.

2.2 Label hierarchy

Each image carries a joint target $[y_1, y_2]$, where $y_1 \in \{0, 1, 2\}$ denotes the haematopoietic lineage and $y_2 \in \{0, \dots, 17\}$ the fine cell type. Because every fine class belongs to exactly one lineage, the levels are related by a fixed, non-learned surjection

$$\pi : \{0, \dots, 17\} \rightarrow \{0, 1, 2\}, \quad y_1 = \pi(y_2). \quad (1)$$

Equation (1) is the structural prior the dissertation exploits: it is known biology, not something the model must discover. Figure 3 gives the tree.

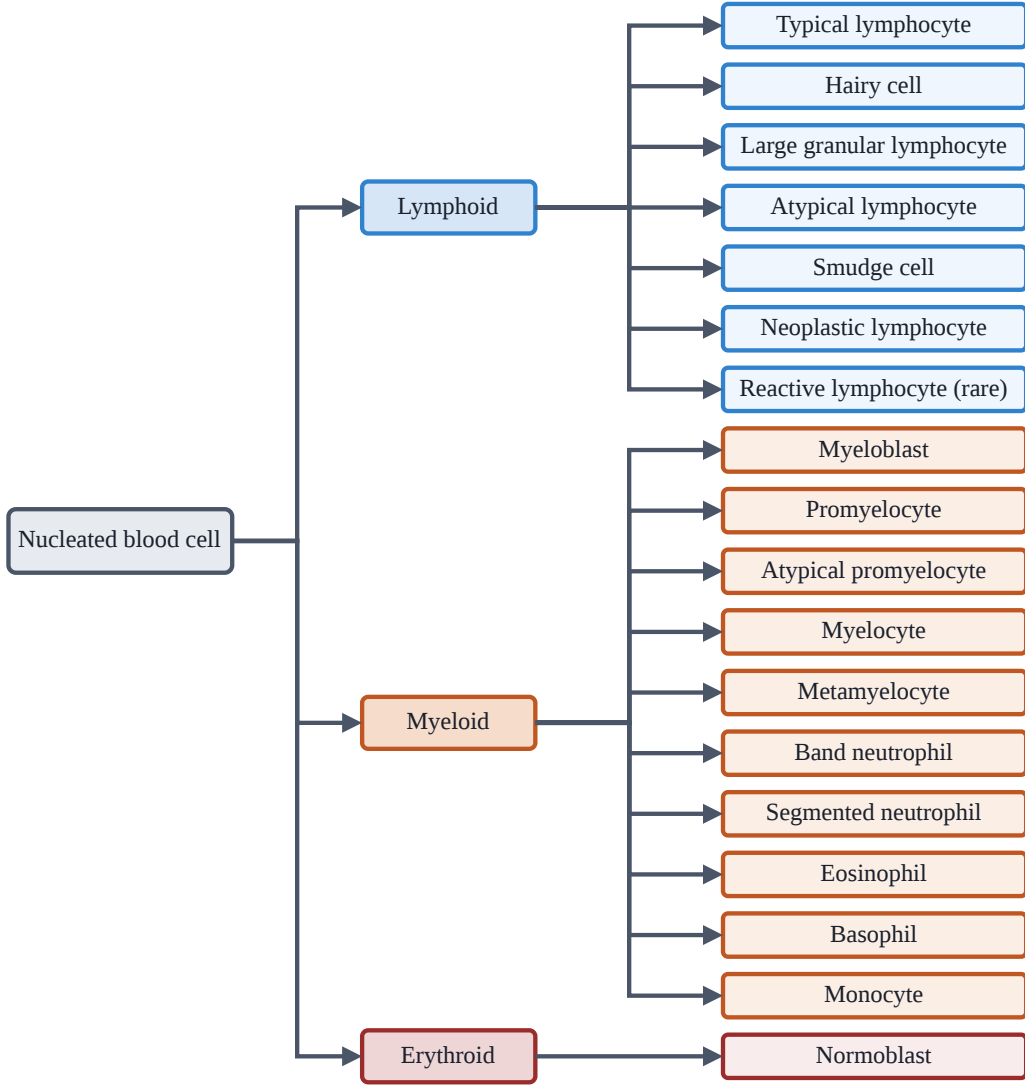


Figure 3: The two-level label hierarchy. Level 1 is the lineage; Level 2 is the fine cell type. Within the myeloid branch the classes follow the maturation continuum.

Fine-class indices are assigned lineage by lineage, and within the myeloid branch they follow the maturation continuum: myeloblast $\rightarrow$ promyelocyte $\rightarrow$ myelocyte $\rightarrow$ metamyelocyte $\rightarrow$ band neutrophil $\rightarrow$ segmented neutrophil. This ordering is preserved in the class indices and in every confusion-matrix axis, so that biologically adjacent stages appear adjacent to the diagonal. Adjacent-stage confusion is the clinically expected error mode, and an alphabetical or frequency-sorted axis would scatter it across the plot and hide it.

2.3 Class imbalance

Table 1 reports the 18 classes with their lineage, corpus counts, and partition sizes. Writing n_c for the number of images in class c , the imbalance ratio is

$$\text{IR} = \frac{\max_c n_c}{\min_c n_c} = \frac{8,606}{33} \approx 261. \quad (2)$$

Eight classes fall at or below a threshold of 1,000 images and are designated *minority* classes throughout. The threshold sits in a natural gap in the distribution — the next class above it holds 1,658 images — so it separates a genuine cluster rather than cutting one in half. Figure 4 plots the distribution on both linear and logarithmic axes; the logarithmic panel is the informative one, since on a linear axis the rarest four classes are visually indistinguishable from zero.

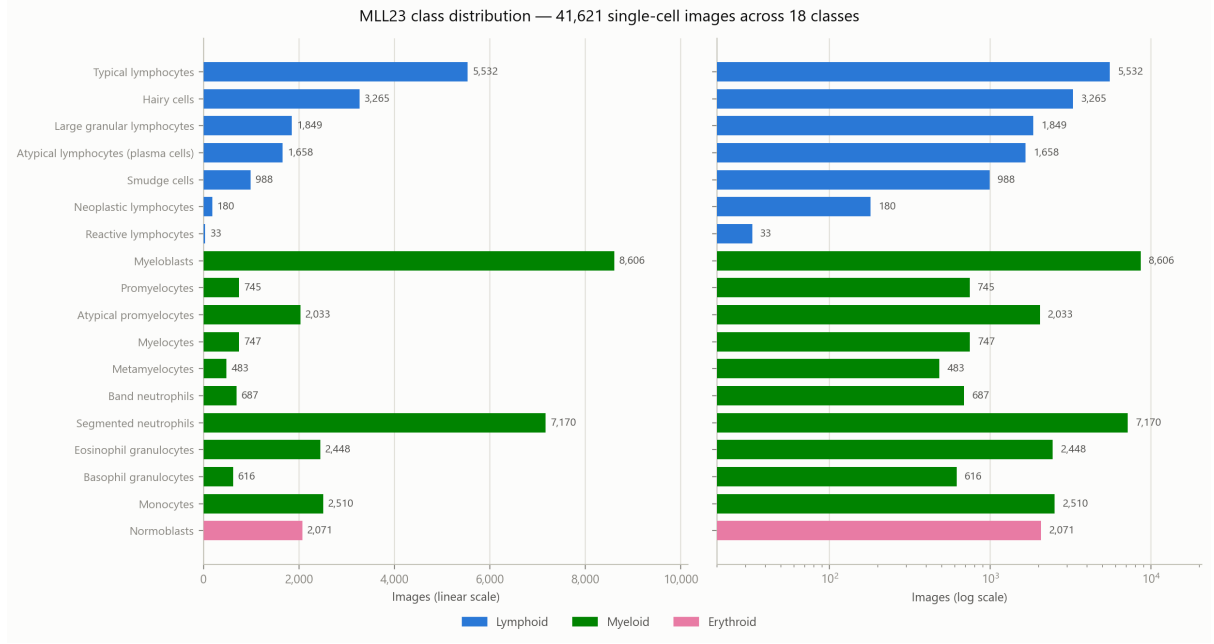


Figure 4: Class distribution on linear (left) and logarithmic (right) axes, coloured by lineage. The 261:1 ratio between myeloblasts and reactive lymphocytes is the condition the training objective must accommodate.

This imbalance is the subject matter of the dissertation, not an obstacle to be removed. It is therefore never resampled away: the training distribution is left intact and the imbalance is addressed in the objective function (Section 5). Resampling would change the question being asked.

Table 1: The 18 MLL23 classes, in class-index order. Minority classes ($n_c \leq 1,000$) are marked †. Partition sizes are the realised counts of the persisted stratified split.

y_2	Cell type	Total	Train	Val	Test
<i>Lymphoid</i> ($y_1 = 0$; 13,505 images)					
0	Typical lymphocytes	5,532	3,872	830	830
1	Hairy cells	3,265	2,285	490	490
2	Large granular lymphocytes	1,849	1,294	277	278
3	Atypical lymphocytes (plasma cells)	1,658	1,161	249	248
4	Smudge cells†	988	692	148	148
5	Neoplastic lymphocytes†	180	126	27	27
6	Reactive lymphocytes†	33	23	5	5
<i>Myeloid</i> ($y_1 = 1$; 26,045 images), in maturation order					
7	Myeloblasts	8,606	6,024	1,291	1,291
8	Promyelocytes†	745	521	112	112
9	Atypical promyelocytes	2,033	1,423	305	305
10	Myelocytes†	747	523	112	112
11	Metamyelocytes†	483	338	73	72
12	Band neutrophils†	687	481	103	103
13	Segmented neutrophils	7,170	5,019	1,075	1,076
14	Eosinophil granulocytes	2,448	1,714	367	367
15	Basophil granulocytes†	616	431	93	92
16	Monocytes	2,510	1,757	376	377
<i>Erythroid</i> ($y_1 = 2$; 2,071 images)					
17	Normoblasts	2,071	1,450	310	311
Total		41,621	29,134	6,243	6,244

3 Data preparation

3.1 Deterministic stratified partitioning

The corpus is partitioned once into training, validation and test sets in 70/15/15 proportion, stratified on the fine label y_2 . Stratifying on y_2 stratifies the lineage automatically, by Equation (1).

The partition is produced by two successive stratified draws under a fixed seed (Algorithm 1) and then persisted to disk. Every experimental arm reads the same persisted file, so the split is identical across arms, seeds and backbones by construction rather than by convention.

Algorithm 1 Deterministic stratified partitioning

Input: manifest D with fine labels y_2 ; fractions $f_{\text{tr}} = 0.70$, $f_{\text{va}} = f_{\text{te}} = 0.15$; seed $s = 42$

Output: each row of D assigned to exactly one of $\{\text{train}, \text{val}, \text{test}\}$

```
1:  $(\mathcal{T}, \mathcal{H}) \leftarrow \text{STRATIFIEDSPLIT}(D, \text{train\_size} = f_{\text{tr}}, \text{strata} = y_2, \text{seed} = s)$ 
2:  $r \leftarrow f_{\text{va}} / (1 - f_{\text{tr}})$   $\triangleright$  share of the held-out remainder going to validation
3:  $(\mathcal{V}, \mathcal{E}) \leftarrow \text{STRATIFIEDSPLIT}(\mathcal{H}, \text{train\_size} = r, \text{strata} = y_2[\mathcal{H}], \text{seed} = s)$ 
4: assign labels train, val, test to  $\mathcal{T}, \mathcal{V}, \mathcal{E}$ 
5: for all classes  $c$  do
6:   for all partitions  $P \in \{\mathcal{T}, \mathcal{V}, \mathcal{E}\}$  do
7:     if  $|\{i \in P : y_2^{(i)} = c\}| = 0$  then
8:       report “class  $c$  absent from  $P$ ”  $\triangleright$  fails loudly rather than degrading silently
9:     end if
10:  end for
11: end for
12: verify no image path occurs in more than one partition
13: verify realised fractions deviate from requested by  $< 0.01$ 
14: persist the assignment
```

The validation loop in Algorithm 1 is necessary because the rarest class holds only 33 images. A 70/15/15 split leaves 23/5/5 — thin but non-empty in every partition, which is the guarantee the design requires. An empty validation or test cell for that class would make its recall undefined and silently distort the macro average, so the condition is asserted rather than assumed.

3.2 Preprocessing

Evaluation preprocessing is deterministic. An image $I \in \mathbb{Z}^{288 \times 288 \times 3}$ is bilinearly resampled to 224×224 , scaled to $[0, 1]$, and standardised channel-wise:

$$\hat{I}^{(k)} = \frac{I_{\downarrow}^{(k)} / 255 - \mu_k}{\sigma_k}, \quad k \in \{R, G, B\}, \quad (3)$$

with ImageNet statistics $\mu = (0.485, 0.456, 0.406)$ and $\sigma = (0.229, 0.224, 0.225)$.

Two choices in Equation (3) deserve comment. First, the input resolution is 224×224 for *every* backbone, including the Vision Transformer. Holding resolution constant is what keeps the backbone comparison controlled; allowing the transformer a larger input would confound architecture with resolution. Second, ImageNet statistics are retained even though the corpus’s own channel means are $(0.742, 0.655, 0.781)$ — brighter than ImageNet in every channel, and widest in blue, reflecting the violet Pappenheim stain against a bright background. After standardisation the data therefore centres near $(1.12, 0.89, 1.67)$ rather than zero. This is intentional: the pretrained backbones expect inputs standardised the way they were trained, and substituting dataset-specific statistics would trade away part of the transfer benefit. The alternative is defensible and is noted as such, but it is not the configuration used here.

3.3 Class-conditional augmentation

Training applies online augmentation before the deterministic pipeline of Equation (3). Augmentation is *class-conditional*: minority classes receive an intensified regime, so that the eight rarest types are presented in greater apparent variety without altering the sampling distribution. For a class c the strength multiplier is

$$s(c) = \begin{cases} 2.0 & \text{if } n_c \leq 1,000 \quad (\text{minority}), \\ 1.0 & \text{otherwise,} \end{cases} \quad (4)$$

applied to the rotation range and jitter magnitude, and capped as shown in Table 2.

Table 2: Augmentation parameters. Geometric transforms are applied at the native 288 px resolution, before the single downsample to 224.

Transform	Majority classes	Minority classes
Horizontal flip	$p = 0.5$	$p = 0.5$
Vertical flip	$p = 0.5$	$p = 0.5$
Random rotation	$\pm 90^\circ$	$\pm 180^\circ$
Brightness / contrast jitter	0.1	0.2
RandAugment	2 ops, magnitude 9	3 ops, magnitude 9

Rotation is label-preserving here in a way it is not for natural images: a blood cell has no canonical orientation, so widening the rotation range toward a full circle adds variety without distorting the class. Rotation is performed at the native 288 px resolution *before* the downsample, so resampling occurs once rather than compounding, and the empty corners produced by rotating a square frame are filled with white — the colour of the smear background — so that the fill introduces no colour the network would not otherwise encounter.

An automated policy (RandAugment) is used in place of the plain jitter in the final protocol. It replaces rather than supplements colour jitter, since stacking two independent colour perturbations would compound them uncontrollably. Figure 5 contrasts the three policies on a single cell.

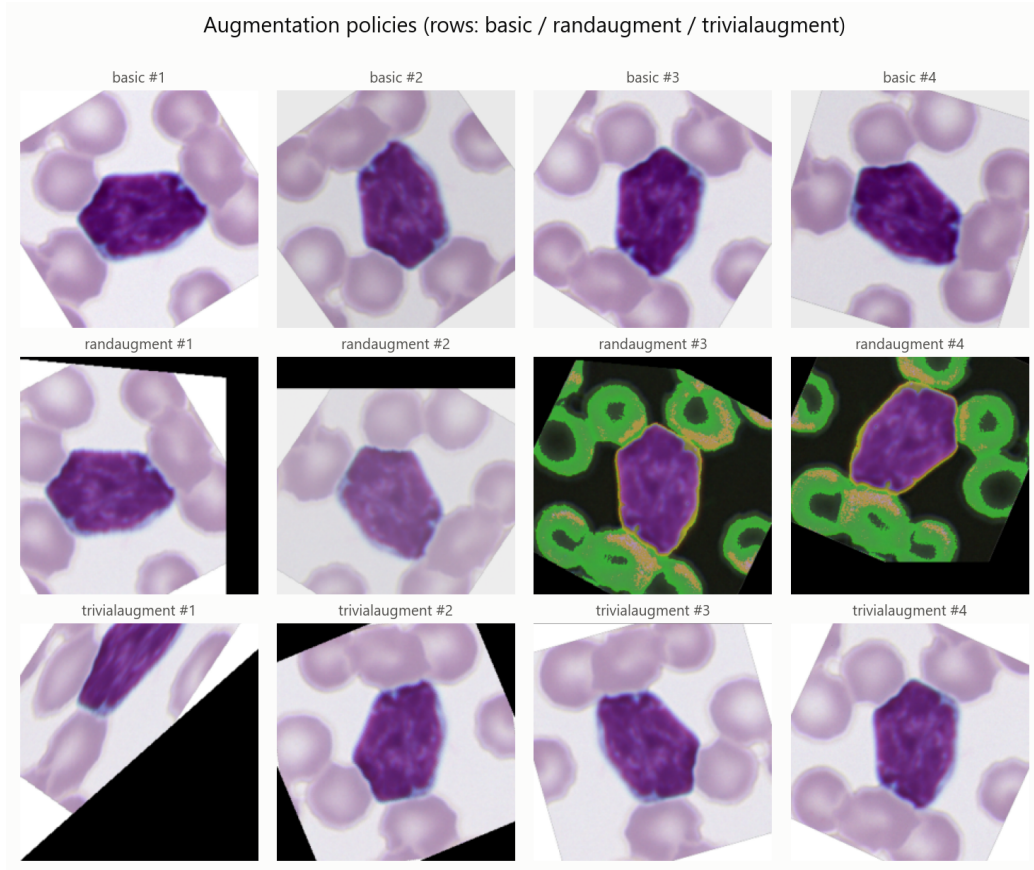


Figure 5: The three augmentation policies applied to one image, four draws each. The basic policy (top) applies flips, white-filled rotation and mild jitter. The automated policies (middle, bottom) are markedly stronger; note that their internal geometric operations fill with black rather than white, and that RandAugment’s colour operations can invert the stain palette entirely.

Figure 5 exposes a caveat that the white-fill argument above does not cover. The white fill is applied to the explicit rotation only; the automated policies perform their own shear, translation and rotation internally, and those default to a black fill. RandAugment’s photometric operations can also invert the stain palette outright, as the third and fourth draws in the middle row show. The resulting images are far outside the distribution of any real smear. This is a recognised property of automated policies rather than a defect — their regularising value comes precisely from the severity of the perturbation, and the measured effect of enabling them was positive — but the “introduces no colour the network would not otherwise see” justification applies strictly to the basic policy. Constraining the automated policies to a white fill is identified as a refinement for future work.

3.4 Stain normalisation

Slide preparation and imaging introduce colour variation unrelated to cell identity. One experimental arm therefore evaluates stain normalisation as an additional preprocessing stage, using

Reinhard’s method: the image is converted to the decorrelated Lab colour space and each channel is matched to a reference by mean and standard deviation,

$$L'_k = (L_k - \bar{L}_k) \frac{\sigma_k^{\text{ref}}}{\sigma_k} + \bar{L}_k^{\text{ref}}, \quad k \in \{l, a, b\}. \quad (5)$$

Reinhard is preferred to Macenko for the training arm on cost grounds: Macenko re-estimates a stain-colour basis per image from the optical-density plane and is roughly twenty times more expensive, which at the measured data-loading throughput would starve the GPU and make that arm slower for reasons unrelated to the science. Macenko is retained for qualitative comparison only.

Critically, the reference statistics in Equation (5) are fitted on the *training split alone*. Fitting on the full corpus would leak validation and test colour statistics into training.

Figure 6 compares both methods against the original images. Macenko produces the more conservative correction, preserving the relative contrast between nucleus and cytoplasm; Reinhard shifts the overall colour balance more aggressively, which is visible in the third row where the background takes on a warmer cast. Whether this difference matters for classification accuracy is an empirical question, and is precisely what the stain-normalisation arm is designed to answer.

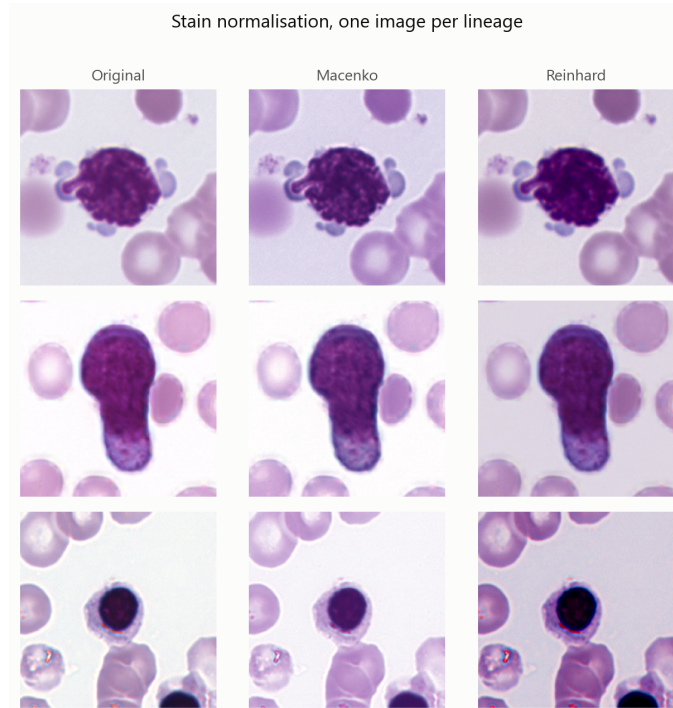


Figure 6: Stain normalisation, one image per lineage. Left: original. Centre: Macenko, which re-estimates a stain-colour basis per image. Right: Reinhard, which matches channel statistics in Lab space and is the method used in the training arm on cost grounds.

3.5 Decoded-image cache

Throughput measurement showed the pipeline to be data-bound rather than compute-bound: decoding TIFFs sustained 178 images/s at four workers, and *fell* to 119 images/s at eight from worker contention, against a full training step of 82 images/s. Decoding was therefore the binding constraint, and backbone choice barely moved wall-clock time.

A cache of decoded images at native 288 px resolution raises throughput to approximately 850 images/s. Caching at 288 rather than at the model input size is deliberate: it preserves the property that rotation precedes the single downsample, so the cache removes decode cost without altering augmentation semantics. A separate cache holds the Reinhard-normalised variant, so that the stain arm runs at the same throughput as the others and a runtime difference between arms never becomes a confound.

4 Model architecture

4.1 Shared backbone with two heads

The model is a pretrained backbone f_θ producing a pooled D -dimensional representation, followed by two linear heads:

$$z = \text{Dropout}(f_\theta(\hat{I})), \quad z \in \mathbb{R}^D, \quad (6)$$

$$\mathbf{u} = W_1 z + b_1, \quad \mathbf{u} \in \mathbb{R}^3 \quad (\text{lineage logits}), \quad (7)$$

$$\mathbf{v} = W_2 z + b_2, \quad \mathbf{v} \in \mathbb{R}^{18} \quad (\text{fine logits}). \quad (8)$$

The two heads are trained jointly on the shared trunk, so the coarse decision regularises the fine one and rare types borrow representational strength from their lineage neighbours. Figure 7 shows the coupling.

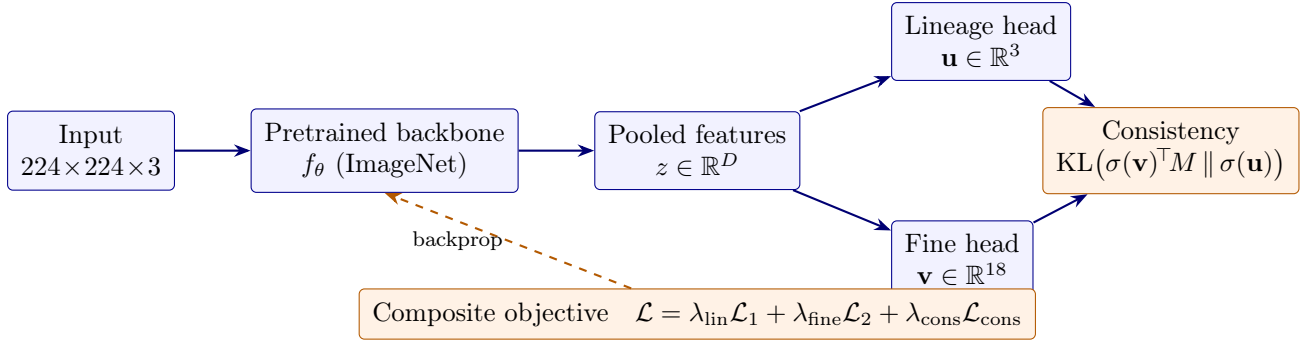


Figure 7: The two-head architecture. Both heads read the same pooled representation; the consistency term marginalises the fine posterior into lineage space through the fixed map M and penalises disagreement with the lineage head. Gradients from all three terms flow into the shared backbone.

A single class implements every arm. In hierarchical mode both heads are constructed and returned; in flat mode the lineage head is never constructed, so it contributes neither parameters nor weight decay — an unused head would make the “identical backbone” claim quietly false. At inference the flat model’s lineage prediction is derived from its fine prediction through Equation (1), which is what makes the hierarchical error decomposition of Section 8.2 computable for the baseline and hence comparable across arms.

4.2 Why the lineage head regularises rather than gates

An obvious alternative architecture is a hard gate: predict the lineage first, then route the sample to one of three per-lineage sub-classifiers. That design was considered and rejected on evidence. Exploratory analysis on frozen ImageNet features measured a lineage silhouette coefficient of approximately zero — the lineages do *not* form separable global clusters — while k -nearest-neighbour lineage agreement was approximately 0.78 against a chance baseline of 0.45. The lineage structure is therefore real but *local*.

A hard gate assumes global separability, which the data rejects, and every gate misfire would produce an unrecoverable cross-lineage error. The soft coupling used instead — a shared trunk plus a consistency penalty — exploits the local structure that exists without asserting the global structure that does not. Figure 8 shows the evidence.

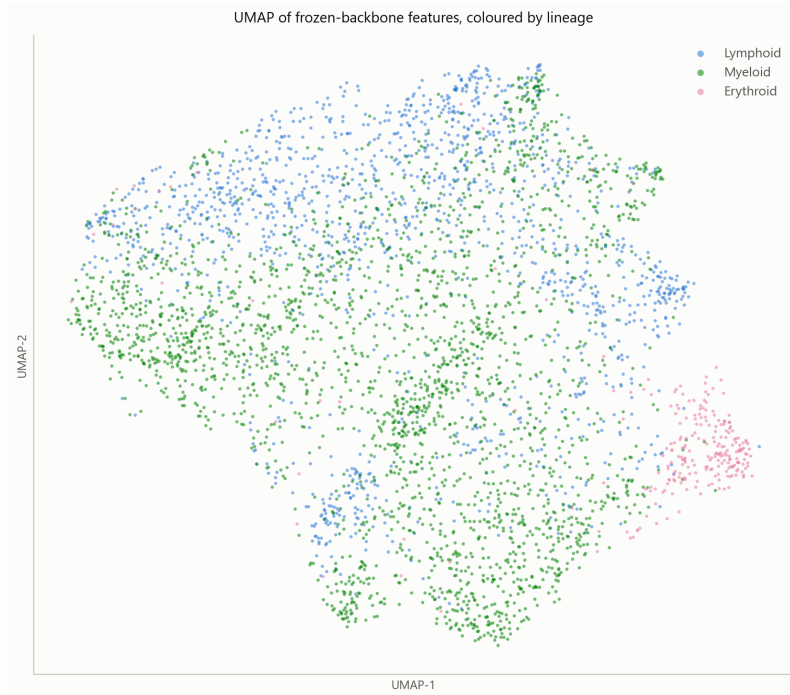


Figure 8: UMAP projection of frozen ImageNet backbone features, coloured by lineage. The lineages interleave rather than forming separable clusters, which is what rules out a hard routing gate. The structure that does exist is local: a cell’s nearest neighbours usually share its lineage even though the lineages form no global cluster.

Two caveats attach to Figure 8. It is a two-dimensional projection, so apparent overlap is weaker evidence than the silhouette coefficient quoted above; and it is computed on *frozen* features, before any fine-tuning. Fine-tuning may well induce more lineage separation than the projection shows. The figure motivates the architectural choice; it does not by itself establish the limits of what the trained model can represent.

4.3 Candidate backbones

Four backbones are compared (Table 3), spanning a lightweight CNN, a standard residual CNN, a modernised CNN, and a transformer. All are initialised from ImageNet weights and fully fine-tuned. The head input width is measured with a single dummy forward pass at construction rather than read from the library’s reported feature count, which is unreliable for some architectures.

Table 3: Candidate backbones. All are fine-tuned end to end at 224×224 .

Key	Architecture	Family
<code>mobilenet</code>	MobileNetV3-Large	Lightweight CNN
<code>resnet</code>	ResNet-50	Residual CNN
<code>convnext</code>	ConvNeXt-Tiny	Modernised CNN
<code>vit</code>	ViT-Small/16	Vision Transformer

5 Training objective

The objective has three components. Two address imbalance; the third couples the label levels.

5.1 Class-balanced weighting

Samples of the same class overlap in feature space, so the n -th sample of a class contributes less new information than the first. Following Cui et al., the *effective number* of samples of a class with n_c images is $(1 - \beta^{n_c})/(1 - \beta)$, and the class weight is its reciprocal:

$$w_c = \frac{1 - \beta}{1 - \beta^{n_c}}, \quad \tilde{w}_c = \frac{w_c}{\frac{1}{C} \sum_j w_j}, \quad \beta = 0.9999. \quad (9)$$

As $\beta \rightarrow 0$ this recovers uniform weighting; as $\beta \rightarrow 1$ it approaches inverse-frequency weighting. The normalisation to unit mean in Equation (9) is essential to the ablation design: without it, the weighted arm would train at a different effective learning rate and the comparison would confound reweighting with step size. The same construction is applied at the lineage level, where the counts are the sums over each lineage’s members — the lineage level is itself imbalanced, myeloid accounting for roughly 63% of the corpus.

5.2 Focal loss

Class weighting addresses *how many* samples a class has; it does not address *how hard* an individual sample is. The focal loss of Lin et al. supplies the second factor, down-weighting confidently correct examples so that the gradient is not dominated by the 8,606 myeloblasts:

$$\mathcal{L}_{\text{focal}}(\mathbf{v}, y) = (1 - p_y)^\gamma \text{CE}(\mathbf{v}, y; \tilde{w}), \quad p_y = \text{softmax}(\mathbf{v})_y, \quad \gamma = 2. \quad (10)$$

The modulating factor p_y in Equation (10) is computed from the *unweighted* posterior. It should reflect the difficulty of the sample, not the rarity of its class; the class weight $\tilde{w}$ already carries rarity, and folding it into both factors would double-count it.

5.3 Hierarchical consistency

The term that makes the objective hierarchical rather than merely multi-task couples the two heads. Let $M \in \{0, 1\}^{18 \times 3}$ be the one-hot matrix of the map π from Equation (1). Right-multiplying a fine posterior by M marginalises it into lineage space, since each fine class belongs to exactly one lineage. The consistency penalty is the Kullback–Leibler divergence between that marginalised belief and the lineage head’s own posterior:

$$\mathcal{L}_{\text{cons}} = \text{KL}\left(\text{softmax}(\mathbf{v})^\top M \parallel \text{softmax}(\mathbf{u})\right). \quad (11)$$

Without Equation (11) the two heads are free to contradict each other and the claim that the coarse decision regularises the fine one has no mechanism behind it. Gradients flow into both heads, pulling them toward agreement rather than forcing one to chase the other.

5.4 Composite objective and ablation switches

The three components combine as

$$\mathcal{L} = \lambda_{\text{lin}} \mathcal{L}_{\text{focal}}(\mathbf{u}, y_1) + \lambda_{\text{fine}} \mathcal{L}_{\text{focal}}(\mathbf{v}, y_2) + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}}, \quad (12)$$

with $\lambda_{\text{lin}} = 0.3$, $\lambda_{\text{fine}} = 1.0$ and $\lambda_{\text{cons}} = 0.1$. The fine term is held at unity because it is the task being evaluated. The consistency weight is deliberately small, for the reason given above: the empirical support is for *local* lineage structure, so consistency is applied as a nudge and not as a constraint. Forcing it hard would impose a global structure the data does not exhibit.

The two ablations are field values on this one object rather than separate loss classes (Table 4), which is what keeps the arms honest.

Table 4: The ablation switches. Both are implemented as parameters of a single loss object, not as alternative implementations.

Switch	Effect	Purpose
<code>use_hierarchy = False</code>	$\lambda_{\text{lin}} = \lambda_{\text{cons}} = 0$	Fine-level loss only; removes the hierarchy
<code>use_imbalance = False</code>	$\tilde{w} \equiv 1$, $\gamma = 0$, no smoothing	Plain cross-entropy; removes the imbalance treatment

5.5 Hierarchical mixing

MixUp and CutMix blend two examples and their labels. Because MLL23 carries two label levels, a mix must remain consistent across levels: the same coefficient λ applies to both targets, so the coarse and fine losses see the same blend. Applying independent mixes per level would let the objective disagree with itself about how much of each example is present. For a batch and a shuffled partner indexed by permutation ρ ,

$$\tilde{x} = \lambda x + (1 - \lambda) x_\rho, \quad \mathcal{L} = \lambda \mathcal{L}(\cdot, y) + (1 - \lambda) \mathcal{L}(\cdot, y_\rho), \quad \lambda \sim \text{Beta}(\alpha, \alpha). \quad (13)$$

Labels are interpolated in the *loss* rather than mixed as hard targets, which keeps the class-balanced weighting of Equation (9) well defined. For CutMix, λ is recomputed from the realised patch area, since integer pixel boundaries rarely match the sampled ratio exactly.

6 Training protocol

6.1 Optimisation

Training uses AdamW with decoupled weight decay, a linear warmup over the first epoch, and cosine annealing thereafter over a *fixed* budget of T total steps:

$$\eta_t = \begin{cases} \eta_0 \frac{t+1}{T_w}, & t < T_w, \\ \frac{\eta_0}{2} \left(1 + \cos \pi \frac{t - T_w}{T - T_w} \right), & t \geq T_w. \end{cases} \quad (14)$$

Warmup matters with pretrained weights: a full-magnitude step into an ImageNet initialisation at the start of training discards much of what transfer learning provides.

Early stopping is disabled, and this is a deliberate correction. The cosine schedule of Equation (14) is defined over the configured budget, so stopping early truncates the anneal and leaves the model mid-training at a high learning rate. An earlier iteration of this study configured 50 epochs but stopped between 12 and 37, leaving runs at up to 86% of peak learning rate; the number of epochs a run survived then correlated with its score at $r = +0.87$, so the arm ranking largely measured the stopping rule rather than the method. A fixed budget guarantees every arm receives an identical anneal.

6.2 Variance reduction

Two mechanisms reduce the noise in the quantity used for model selection. An exponential moving average of the weights is maintained,

$$\theta_t^{\text{EMA}} = d \theta_{t-1}^{\text{EMA}} + (1 - d) \theta_t, \quad d = 0.999, \quad (15)$$

and the averaged weights — not the live ones — are evaluated, selected and reported. At test time, predictions are averaged over the four horizontal and vertical flip combinations; since blood cells have no canonical orientation, these are genuine label-preserving equivalents rather than distortions.

The justification is measured. Raw epoch-to-epoch validation macro F1 varies with standard deviation 0.021, which is *larger* than the between-arm differences under study. Without smoothing, checkpoint selection would be closer to a lottery than a decision. Together these mechanisms reduce that variation to 0.0097, a 54% reduction.

6.3 Numerical precision

Mixed precision uses bfloat16 rather than float16. A float16 run reached validation macro F1 of 0.80 and then diverged to NaN at the third epoch, wasting the remainder of the run — NaN weights never recover. bfloat16 retains float32’s exponent range and so cannot overflow in the same way. Gradient scaling, which exists to prevent float16 gradient underflow, is correspondingly disabled. The loss itself is computed in float32 regardless of the surrounding autocast context, because the class-balanced weights span a wide range and the focal term exponentiates a log-probability; both are numerically delicate in reduced precision.

6.4 The training loop

Table 5 lists the hyperparameters, which are held identical across every arm. Algorithm 2 states one training run. Model selection is on validation macro F1 and never on accuracy: at an imbalance ratio of 261, a classifier that ignores reactive lymphocytes entirely forfeits under 0.1% of accuracy, so an accuracy-selected checkpoint is precisely one that has learned to abandon the classes the study is about.

Algorithm 2 One training run

Input: configuration κ ; persisted split; training class counts n

Output: checkpoint selected on best validation macro F1

```

1: seed Python, NumPy and Torch from  $\kappa$ .seed
2: build model in mode  $\kappa$ .mode; build  $\mathcal{L}$  from  $n$ ,  $\kappa$ .use_hierarchy,  $\kappa$ .use_imbalance
3:  $\theta^{\text{EMA}} \leftarrow \theta$ ;  $F^* \leftarrow -\infty$ 
4: for epoch = 1 to  $\kappa$ .epochs do
5:   for all batches  $(x, y_1, y_2)$  do
6:      $(\tilde{x}, y_a, y_b, \lambda) \leftarrow \text{MAYBEMIX}(x, y_1, y_2)$  ▷ one  $\lambda$  shared by both levels
7:      $(\mathbf{u}, \mathbf{v}) \leftarrow$  forward pass under bfloat16 autocast
8:      $\ell \leftarrow \lambda \mathcal{L}(\cdot, y_a) + (1 - \lambda) \mathcal{L}(\cdot, y_b)$  ▷ computed in float32
9:     if  $\ell$  is not finite then
10:      skip the optimiser step ▷ never propagate NaN into the weights
11:     else
12:       backpropagate; step AdamW; step the schedule of Eq. (14)
13:       update  $\theta^{\text{EMA}}$  by Eq. (15)
14:     end if
15:   end for
16:    $F \leftarrow$  validation macro F1 of  $\theta^{\text{EMA}}$ 
17:   if  $F > F^*$  then
18:      $F^* \leftarrow F$ ; checkpoint  $\theta^{\text{EMA}}$ 
19:   end if
20:   append epoch metrics to the run history
21: end for
22: evaluate the selected checkpoint on the test split with 4-view TTA
```

Table 5: Training hyperparameters, held identical across all arms.

Parameter	Value
Optimiser	AdamW
Learning rate η_0	3×10^{-4}
Weight decay	0.05
Batch size	64
Epochs	30 (fixed; no early stopping)
Warmup	1 epoch, linear
Schedule	Cosine annealing
Label smoothing	0.05
Focal γ	2.0
Class-balanced β	0.9999
Loss weights	$\lambda_{\text{lin}} = 0.3, \lambda_{\text{fine}} = 1.0, \lambda_{\text{cons}} = 0.1$
Augmentation policy	RandAugment (class-conditional)
Batch mixing	CutMix, $p = 0.5, \alpha = 1.0$
Weight EMA decay	0.999
Test-time augmentation	4 flip views
Mixed precision	bfloat16
Model selection	Validation macro F1

7 Experimental design

7.1 The arms

Five experiments are required, under identical splits and backbones. They reduce to four distinct configurations (Table 6), because the hierarchy ablation applied to the proposed model *is* the flat baseline: removing the lineage head and its loss terms from the two-head model leaves a single-head 18-class classifier. That configuration is trained once and reported twice. The alternative — training the identical configuration under two names — would consume a fifth of the compute budget to produce two draws from the same distribution and invite the reader to mistake them for independent evidence.

Table 6: The four distinct configurations and the five reported experiments.

Arm	Mode	Hierarchy	Imbalance	Stain	Reported as
<code>flat_baseline</code>	flat	—	✓	—	1. Flat baseline; 4. Hierarchy ablation
<code>hierarchical</code>	hier	✓	✓	—	2. Full hierarchical model (proposed)
<code>no_imbalance</code>	hier	✓	—	—	3. Imbalance ablation
<code>stain_norm</code>	hier	✓	✓	✓	5. Stain-normalised input

7.2 Two phases

Phase 1 screens the four backbones of Table 3 under the hierarchical configuration for a short fixed budget at one seed, and selects the backbone carried into Phase 2 on validation macro F1. This is a *selection* run and not the backbone comparison result: a short budget need not rank

backbones the way a full run would, and the write-up states this as a limitation rather than presenting the screen as evidence.

Phase 2 trains the four configurations of Table 6 at three seeds each on the selected backbone, giving twelve runs. The matrix is resumable: a run already recorded in the results file is skipped, so an interruption costs only the unfinished runs.

8 Evaluation

8.1 Metrics

Plain accuracy is explicitly rejected as a headline number. At an imbalance ratio of 261 it cannot see the behaviour under study. The headline metrics are macro F1 and balanced accuracy, which weight every class equally regardless of frequency:

$$\text{macro-}F_1 = \frac{1}{C} \sum_{c=1}^C \frac{2P_c R_c}{P_c + R_c}, \quad \text{balanced accuracy} = \frac{1}{C} \sum_{c=1}^C R_c, \quad (16)$$

where P_c and R_c are the precision and recall of class c . Accuracy, macro precision, macro recall and the full confusion matrix are also reported. Where a rare class receives no predictions in an early epoch its score is taken as zero rather than undefined, since zero is the correct value and a NaN would propagate into the macro average.

A minority-class view reports the same quantities restricted to samples whose true class is one of the eight minority classes. This is recall-oriented by construction — it answers how many of the rare cells present were found and correctly named — and is labelled as such rather than presented as an overall score.

8.2 Hierarchical error decomposition

Every prediction is assigned to exactly one of three outcomes, using Equation (1) to derive lineages from fine labels:

$$\text{outcome}(y, \hat{y}) = \begin{cases} \text{correct,} & \hat{y} = y, \\ \text{within-lineage error,} & \hat{y} \neq y \wedge \pi(\hat{y}) = \pi(y), \\ \text{cross-lineage error,} & \pi(\hat{y}) \neq \pi(y). \end{cases} \quad (17)$$

The diagnostic of interest is the share of *errors* that remained inside the correct lineage,

$$\text{within-error share} = \frac{\#\{\text{within-lineage errors}\}}{\#\{\text{errors}\}}. \quad (18)$$

Equation (18) is the direct test of the dissertation’s thesis. A lineage-aware model should shift errors from cross-lineage to within-lineage even where total error is unchanged, and the two are not clinically equivalent: confusing a myelocyte with a metamyelocyte — adjacent maturation stages — is mild, whereas calling a lymphocyte a myeloblast is severe. No scalar accuracy measure can express this distinction.

8.3 Saliency

Grad-CAM produces class-discriminative saliency maps by weighting the activation maps $A^{(k)}$ of a chosen layer by the gradient of the target logit y^c pooled over space:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^{(k)}}, \quad L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^{(k)}\right). \quad (19)$$

Saliency is a deliverable of this study rather than an afterthought: the maps are used to verify that the network attends to the nucleus, chromatin texture and cytoplasm rather than to background artefacts.

Applying Equation (19) to a Vision Transformer requires care. The classifier reads only the class token, so at the final block the patch tokens no longer influence the output and their gradients are exactly zero — yielding a uniformly blank map that min-max normalisation renders as a plausible-looking figure. Saliency is therefore computed at the input to the final block’s attention, where patch tokens still route into the class token, and the implementation raises an error if no gradient was captured rather than returning a blank map. Figure 9 illustrates the output of the procedure.

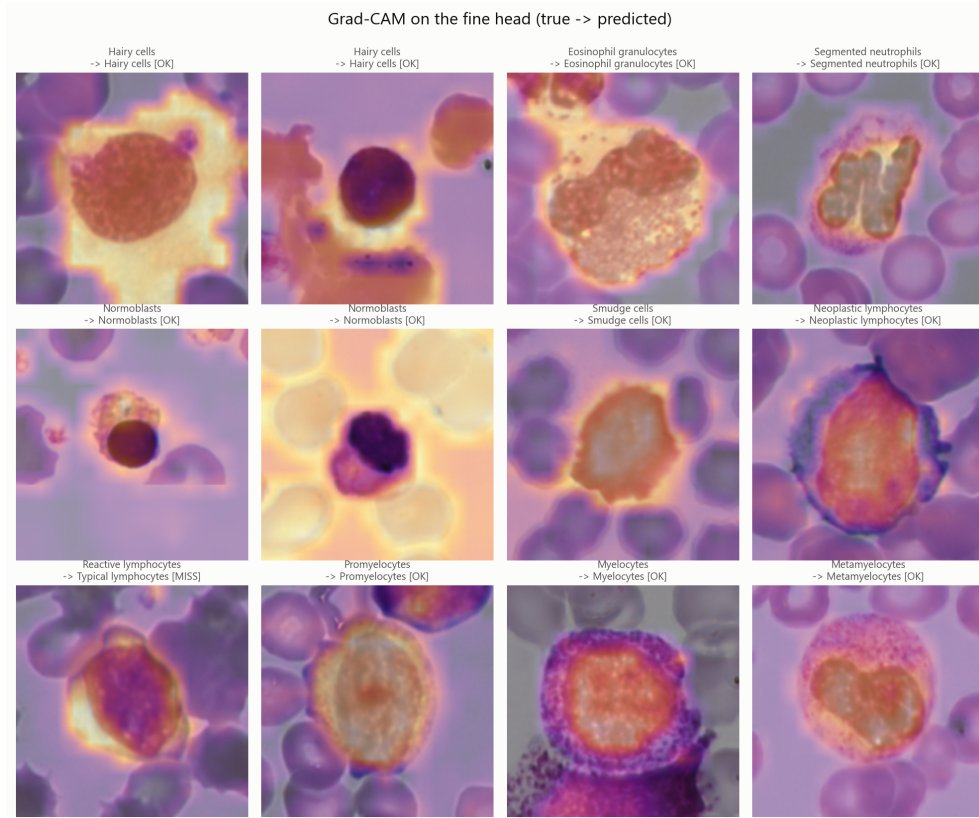


Figure 9: Grad-CAM saliency for the fine head, with true and predicted labels. Attention concentrates on the cell body — nucleus, chromatin texture and cytoplasmic boundary — rather than on surrounding erythrocytes or background, which is the qualitative check the method is included to perform. The figure illustrates the technique; the saliency analysis proper belongs to the results chapter.

8.4 Statistical testing

Because the design uses one fixed partition rather than k folds, paired samples are obtained by retraining each arm at three seeds on that fixed partition and pairing arms by seed. This keeps the split identical across arms — which the ablation design depends on — at the cost of measuring training variance rather than data variance. For paired differences d_i across n seeds,

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \quad d_z = \frac{\bar{d}}{s_d}. \quad (20)$$

Effect sizes are reported alongside p -values throughout. With three seeds the test carries two degrees of freedom, which is thin, and reporting d_z and the per-seed values prevents the reader from being asked to lean on the p -value alone.

9 Reproducibility and threats to validity

Determinism is enforced at three points: the partition is seeded and persisted, each run seeds Python, NumPy and Torch from its configuration, and every configuration is written to the results

file alongside its metrics so that a recorded run can be reconstructed exactly. Correctness is enforced by assertions rather than by inspection — the class hierarchy validates its own invariants at import, downloaded counts are checked against the published figures, and the partition is checked for empty cells, duplicates and proportion drift.

Three limitations are acknowledged. First, three seeds is a thin basis for a paired test; it is mitigated by reporting effect sizes and per-seed values, but not eliminated. Second, the rarest class contributes roughly 23 training images, so its per-class F1 will be unstable across seeds; it is reported rather than concealed inside the macro average. Third, the backbone screen uses a short budget and selects rather than compares, so it does not license claims about the relative merits of the four architectures.